

中国科学技术大学

本科毕业论文



随机特征方法求解高维抛物型方程

作者姓名：	刘行
学 号：	PB22000150
专 业：	信息与计算科学
导 师：	陈景润 教授
完成时间：	2026 年 5 月 7 日

中国科学技术大学本科毕业论文学术诚信承诺书

本人郑重承诺：所呈交的本科毕业论文，是本人在导师指导下进行研究工作所取得的成果。除已特别加以标注和致谢的地方外，论文中不包含任何他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的贡献均已在论文中作了明确的说明。如出现侵犯他人知识产权的行为，由本人承担相应法律责任。本人承诺不存在抄袭、伪造、篡改、代写、买卖毕业论文(设计)等违纪行为。

作者签名：_____

签字日期：_____

摘要

高维偏微分方程 (Partial Differential Equations, PDEs) 在金融, 量子物理, 控制问题等领域都具有重要应用. 然而, 维度诅咒使得传统数值方法在高维情形下计算复杂度呈指数增长, 严重限制了其适用范围. 近年来, 基于机器学习的函数逼近方法为高维问题提供了新的思路. 随机特征法 (Random Feature Method, RFM) 是一类通过构造随机特征基函数对目标函数进行线性表示的方法, 其核心思想是利用随机特征映射将隐式定义的核函数转化为特征函数的数学期望, 并通过有限样本平均进行近似, 从而避免显式构造高维 Gram 矩阵, 将问题转化为低维最小二乘问题. 本文提出一种结合随机特征法与蒙特卡洛采样 (Monte Carlo sampling, MC sampling) 的数值方法, 用于求解高维线性与半线性抛物型 PDEs. 该方法利用随机采样处理高维空间积分, 同时借助随机特征表示降低函数逼近复杂度, 在计算效率与精度之间取得平衡. 首先将方程转化为相应的倒向随机微分方程 (Backward Stochastic Differential Equations, BSDEs), 并确定待求解的目标点. 然后从目标点出发进行蒙特卡洛采样, 并学习解函数在路径点上的梯度及目标点处的函数值, 其中梯度项采用随机特征映射进行逼近. 在线性方程中, 终端匹配问题可化为带正则化的线性最小二乘问题; 在半线性方程中, 本文将终端残差写成非线性最小二乘形式, 并采用 Levenberg-Marquardt (LM) 方法求解. 数值实验包括线性的热方程, Black-Scholes 方程, 以及半线性的 Hamilton-Jacobi-Bellman 方程 (HJB equation) 和 Allen-Cahn 方程. 实验结果表明, 该方法在所测试的高维算例中可以用较短时间得到接近参考值的初值.

关键词: 高维偏微分方程; 随机特征法; 蒙特卡洛方法; 倒向随机微分方程; 最小二乘法

ABSTRACT

High-dimensional partial differential equations (PDEs) arise in a range of applications, including finance, quantum physics, and control theory. However, the curse of dimensionality causes the computational complexity of traditional numerical methods to grow exponentially with respect to the dimension, severely limiting their applicability. In recent years, machine learning-based function approximation methods have provided new perspectives for tackling high-dimensional problems. The Random Feature Method (RFM) represents a class of approaches that approximate target functions via linear combinations of randomly generated feature functions. Its key idea is to employ random feature mappings to transform implicitly defined kernel functions into expectations over feature functions, which are then approximated by finite sample averages. This avoids the explicit construction of high-dimensional Gram matrices and reduces the problem to a low-dimensional least-squares formulation. In this work, a numerical method combining the Random Feature Method with Monte Carlo sampling is proposed for solving high-dimensional linear and semilinear parabolic PDEs. The method leverages random sampling to handle high-dimensional integrals, while using random feature representations to reduce the complexity of function approximation, achieving a balance between computational efficiency and accuracy. The PDE is first reformulated as the corresponding backward stochastic differential equation (BSDE), and a target point is specified for evaluation. Monte Carlo sampling is then performed starting from the target point, and both the gradient of the solution along sampled paths and the function value at the target point are learned. The gradient term is approximated using random feature mappings. In linear equations, the terminal matching problem can be formulated as a linear least squares problem with regularization; In semilinear equations, this paper expresses the terminal residual in a nonlinear least-squares form and employs the Levenberg-Marquardt (LM) method for solving it. Numerical experiments on the linear heat equation, the Black-Scholes equation, the semilinear HJB equation, and the Allen-Cahn equation show that the proposed method can produce initial values close to reference values within short running times on the tested high-dimensional examples.

Key Words: High-dimensional PDEs; Random feature method; Monte Carlo method; Backward stochastic differential equations; Least-squares method

目 录

第一章 引言	4
第一节 研究背景与意义	4
第二节 抛物型方程的基本类型	4
一、线性抛物型方程	5
二、半线性抛物型方程	6
第三节 随机特征方法	9
第四节 深度倒向随机微分方程方法	10
第五节 本文主要工作与章节安排	11
第二章 求解高维偏微分方程的随机特征方法	13
第一节 整体思路	13
第二节 样本路径与随机特征的构造	14
第三节 线性方程的求解	15
第四节 半线性方程的求解	16
第五节 测试阶段与误差评估	18
第三章 数值实验	19
第一节 实验设置	19
第二节 线性方程	20
第三节 半线性方程	21
第四节 参数敏感度分析	23
一、样本数的影响	23
二、随机特征宽度的影响	24
三、维度与运行时间	25
四、随机种子稳定性	26
第五节 Deep BSDE 对比实验	27
第六节 本章小结	28
第四章 总结与展望	29
参考文献	30
附录 A 迭代格式推导	32
第一节 从 PDE 到 BSDE	32

第二节 线性方程终端格式的展开	32
第三节 半线性方程的灵敏度递推	33
附录 B 参考值的计算与可靠性说明	35
第一节 Heat 方程的显式参考值	35
第二节 HJB-LQ 方程的积分参考值	35
第三节 Black-Scholes 方程的 Monte Carlo 参考值	36
第四节 Allen-Cahn 方程的有限差分参考值	37
致谢	39
实验算法伪代码	41
第五节 线性方程求解流程	41
第六节 半线性方程求解流程	42
第七节 测试误差评估流程	43

插图和附表清单

图 3.1	半线性方程的 LM 迭代残差下降	23
图 3.2	样本数对泛化比值和初值相对误差的影响	24
图 3.3	随机特征宽度对测试残差和初值相对误差的影响	24
图 3.4	维度变化下的运行时间和初值相对误差	26
图 3.5	不同随机种子下的初值相对误差	26
图 3.6	本文方法与 Deep BSDE 对比方法的初值相对误差	27
表 3.1	主要实验配置	20
表 3.2	线性方程主要结果	21
表 3.3	半线性方程主要结果	22
表 3.4	半线性方程的 LM 迭代结果	23
表 3.5	样本数变化结果	23
表 3.6	维度变化的平均结果	25
表 3.7	随机种子稳定性结果	26
表 3.8	本文方法与 Deep BSDE 对比方法在半线性方程上的结果	27
表 B.1	Heat 方程参考值	35
表 B.2	HJB-LQ 方程参考值	36
表 B.3	Black-Scholes 方程 Monte Carlo 参考值	37
表 B.4	Allen-Cahn 方程有限差分参考值	38

第一章 引言

第一节 研究背景与意义

偏微分方程 (Partial Differential Equations, PDEs) 是处理各种物理, 金融和控制问题的重要工具. 近年来计算机硬件和计算科学的发展使得低维的简单问题基本可以快速求解. 但是由于传统的有限元方法 (Finite Element Method, FEM) 在解决高维度问题时网格数量呈指数增长, 时间复杂度随之爆炸, 出现了人们常说的维度诅咒 (curse of dimensionality) 现象, 因此需要找到更高效的方法求解高维 PDEs. 这一困难也是近年来高维 PDE 数值算法研究的主要出发点之一^[1-2]. 蒙特卡洛采样 (Monte Carlo sampling, MC sampling) 通过随机路径或随机配置点避免全空间网格生成, 而神经网络等函数逼近工具则为高维样本点上的未知函数表示提供了新的选择. 近年来, 围绕高维或无网格 PDE 求解, 已经发展出 Deep BSDE, Deep Ritz, DGM, PINN, 弱对抗网络等多类神经网络方法, 也有工作讨论了利用深度网络处理复杂几何区域上的 PDE 问题^[3-7]. 在这些方法中, 随机采样和拟蒙特卡洛采样常被用于近似高维积分型损失并避免显式网格构造^[8-10]. 与 PINN 相关的软件实现也逐渐成熟, 例如 DeepXDE 对多类深度学习微分方程方法进行了统一封装^[11]. 近年来, 机器学习 (Machine Learning, ML) 在高维函数逼近中的表现引起了科学计算和计算数学领域的关注. 相关综述指出, 传统有限差分方法 (Finite Difference Method, FDM), FEM 和谱方法 (Spectral Method, SM) 已经能够常规处理三维空间加时间的问题, 稀疏网格方法可以进一步将可处理维度提高到约 8 到 10 维, 但对于上百维 PDE, 传统网格型方法仍然难以直接应用^[12]. 对于更一般的高维 fully nonlinear PDE, 还常需借助二阶 BSDE 等更复杂的随机表示框架, 求解难度会进一步增加^[13]. 因此, 利用深度学习或其他随机化函数逼近方法求解高维 PDE, 成为近几年高维数值计算中的一个重要方向. 本文则在 BSDE 求解框架下使用随机特征法 (Random Feature Method, RFM) 代替常规神经网络, 以降低参数优化的复杂度.

第二节 抛物型方程的基本类型

抛物型方程是一类重要的偏微分方程, 其典型形式为

$$\frac{\partial u}{\partial t} + \mathcal{L}u + f = 0, \quad (1.1)$$

其中 $\mathcal{L}$ 为关于空间变量的二阶线性椭圆算子. 该类方程通常用于描述随时间演化的扩散过程, 例如热传导, 概率密度演化以及金融衍生品定价等问题. 为了方便

与倒向随机微分方程对齐, 本文将椭圆算子拆分得到如下形式:

$$\begin{aligned} \frac{\partial u}{\partial t}(t, x) + \frac{1}{2} \text{Tr}(\sigma \sigma^T(t, x) \nabla_x^2 u(t, x)) \\ + \langle \mu(t, x), \nabla u(t, x) \rangle \\ + f(t, x, u(t, x), \sigma^T(t, x) \nabla u(t, x)) = 0. \end{aligned} \quad (1.2)$$

一、线性抛物型方程

首先考虑其中的线性方程, 即 f 是 $u, \nabla u$ 的线性函数. 一方面, 从数学结构上看, 线性抛物型方程具有良好的正则性与稳定性, 其解通常可以通过解析表示 (如基本解或 Green 函数) 或概率表示 (如 Feynman–Kac 公式) 来刻画, 可以作为 baseline 用来判断方法的正确性并评估计算效率. 另一方面, 线性方程也可以像半线性方程那样通过 Itô 公式变换为 BSDE, 使该路径采样框架能够较自然地推广到非线性情形. 下面列举了常见的线性抛物型方程:

1. 在本文的统一半线性抛物型方程框架下, 热方程对应的随机过程为标准 d 维布朗运动, 即漂移项为零, 扩散系数为单位矩阵.
2. 因此有

$$\mu(t, x) = 0, \quad \sigma(t, x) = I_d, \quad f(t, x, u, z) = 0.$$

代入统一形式后得到

$$\frac{\partial u}{\partial t}(t, x) + \frac{1}{2} \Delta u(t, x) = 0, \quad u(T, x) = g(x).$$

热方程是最经典的线性抛物型偏微分方程之一, 最初来源于热传导问题, 也广泛用于描述粒子扩散, 浓度传播以及随机扰动在空间中的扩散过程. 从概率角度看, 热方程与布朗运动具有直接联系, 其解可以表示为布朗运动终端函数的期望. 因此, 热方程不仅是扩散模型的基础方程, 也是检验随机方法求解线性抛物型 PDE 的标准基准问题. 在高维情形下, 热方程的维度对应空间变量的维数, 即 $x \in \mathbb{R}^d$ 中的 d . 该算例主要用于验证随机特征法在纯扩散线性问题上的基本逼近能力.

3. 在统一框架下, 多资产 Black–Scholes 方程对应

$$\mu(t, x) = r x, \quad \sigma(t, x) = \text{diag}(x) \Sigma^{1/2}, \quad f(t, x, u, z) = -r u.$$

代入统一形式后得到

$$\begin{aligned} \frac{\partial u}{\partial t}(t, x) + \frac{1}{2} \text{Tr}(\text{diag}(x) \Sigma \text{diag}(x) \nabla_x^2 u(t, x)) + \langle r x, \nabla_x u(t, x) \rangle - r u(t, x) = 0, \\ u(T, x) = g(x). \end{aligned}$$

Black–Scholes 方程来源于金融衍生品定价理论, 是数学金融中最具有代表性的线性抛物型方程之一. 该模型将期权价格视为标的资产价格与时间的

函数, 并通过无套利思想和动态对冲方法推导出相应的定价方程; 在经典单资产情形下, 它还给出了欧式期权的封闭形式定价公式^[14]. 对低维 Black-Scholes 方程, 高阶有限差分等传统离散方法仍然可以取得较高精度; 但当收益依赖多个标的资产时, PDE 的空间维度会随资产数量增加而迅速升高^[15]. 在多资产情形下, 若期权收益依赖于多个标的资产, 则 PDE 的空间维度等于资产数量, 因而 Black-Scholes 方程自然成为高维 PDE 方法中的典型应用算例. 近年来, 多资产 Black-Scholes 方程也常被用于测试面向高维金融定价问题的变分或学习型算法^[16]. 设 $x = (x_1, \dots, x_d)$ 表示 d 个资产价格, r 表示无风险利率, Σ 表示资产收益率的协方差矩阵. 在风险中性测度下, 资产价格通常服从几何布朗运动, 其漂移项为 rx , 扩散项与当前资产价格成正比. 其中 $-ru$ 项表示未来收益在无风险利率下的折现效应. 该算例用于检验本文方法在具有乘性扩散, 非零漂移和折现项的高维金融定价问题上的表现.

4. Ornstein-Uhlenbeck 方程对应的随机过程是一类典型的均值回复过程, 常用于描述带阻尼的随机运动, 也广泛出现在统计物理, 金融数学和随机控制等领域. 与布朗运动不同, Ornstein-Uhlenbeck 过程的漂移项会将状态变量拉回长期均值附近, 因而适合描述具有稳定中心或长期均衡水平的随机系统. 在高维情形下, 其维度同样对应状态变量的空间维数.

线性反应-扩散方程用于描述扩散过程与局部反应过程同时存在的现象. 其中扩散项刻画物质, 能量或信息在空间中的传播, 反应项刻画局部增长, 衰减或外部源项作用. 这类方程在化学反应, 生物种群动力学, 热源扩散以及物理场演化问题中都有广泛应用.

尽管部分方程在物理建模中通常定义于低维空间 (如三维), 但在金融建模, 多因子随机过程等问题中, 其状态变量维度可以显著增加, 因此常被用作高维数值算法的基准问题. 由于前两者已经能覆盖纯扩散型和金融乘性扩散型两类典型情形, 且后面还有非线性方程的实验, 本论文将仅对热方程和 Black-Scholes 方程进行数值实验.

二、半线性抛物型方程

如果 f 不是 u 与 ∇u 的线性函数, 则整个方程为半线性抛物型方程. 尽管半线性方程解的解析表示不那么明显, 但通常也有较为完善的理论对其数值做近似估计. 在完成线性方程求解的理论分析与实验验证后, 对半线性方程的实验可以进一步检验该方法的适用范围. 作为线性的推广, 本文依然通过 Itô 公式将此类方程变换为 BSDE 进行求解. 下面列举了常见的半线性抛物型方程:

1. 本文采用线性二次型高斯控制问题对应的 Hamilton-Jacobi-Bellman 方程

(Hamilton–Jacobi–Bellman equation, HJB equation). 在统一半线性抛物型方程框架下, 取

$$\mu(t, x) = 0, \quad \sigma(t, x) = \sqrt{2}I_d,$$

此时

$$z = \sigma^T \nabla_x u = \sqrt{2} \nabla_x u.$$

若控制强度参数为 $\lambda > 0$, 生成元取为

$$f(t, x, y, z) = -\frac{\lambda}{2} \|z\|^2,$$

则得到具体方程

$$\frac{\partial u}{\partial t}(t, x) + \Delta u(t, x) - \lambda \|\nabla_x u(t, x)\|^2 = 0, \quad u(T, x) = g(x).$$

Hamilton–Jacobi–Bellman 方程来源于动态规划和随机最优控制理论. 在最优控制问题中, 解函数通常表示从当前状态出发所能达到的最优代价, 因而也称为值函数. 当状态变量维数较高时, HJB 方程的维度等于控制系统的状态空间维度, 传统网格方法会迅速受到维数灾难的限制. 因此, HJB 方程是检验高维 PDE 方法的重要非线性算例. 例如 Deep BSDE 方法和高维 PINN 方法都将 HJB 方程作为代表性测试问题^[1,10]. 从控制问题角度看, 可设状态变量 $X_t \in \mathbb{R}^d$ 满足受控随机微分方程

$$dX_t = 2\sqrt{\lambda} m_t dt + \sqrt{2} dW_t,$$

其中 m_t 为控制变量, $\lambda > 0$ 表示控制强度, W_t 为 d 维标准布朗运动. 控制目标为最小化代价泛函

$$J(\{m_t\}_{0 \leq t \leq T}) = \mathbb{E} \left[\int_0^T \|m_t\|^2 dt + g(X_T) \right].$$

由动态规划原理可得到上面的 HJB 方程. 该算例主要用于检验本文方法处理梯度型非线性项的能力.

2. 在统一半线性抛物型方程框架下, Allen–Cahn 方程可取

$$\mu(t, x) = 0, \quad \sigma(t, x) = \sqrt{2}I_d,$$

并令

$$f(t, x, y, z) = y - y^3.$$

代入统一形式后得到终端值问题

$$\frac{\partial u}{\partial t}(t, x) + \Delta u(t, x) + u(t, x) - u^3(t, x) = 0, \quad u(T, x) = g(x).$$

Allen–Cahn 方程是一类典型的非线性反应–扩散方程, 常用于描述相分离, 界面演化以及有序–无序转变等物理现象. 其核心特点是同时包含扩散项和

双势阱型反应项. 扩散项刻画空间传播或平滑效应, 非线性反应项刻画系统向不同稳定相的演化趋势. 与 HJB 方程不同, Allen–Cahn 方程的非线性主要依赖于解函数 u 本身, 而不是梯度. 对应的正向时间形式可写为

$$\frac{\partial v}{\partial s}(s, x) = \Delta v(s, x) + v(s, x) - v^3(s, x), \quad v(0, x) = g(x).$$

为了与本文采用的终端值问题形式一致, 令

$$u(t, x) = v(T - t, x),$$

对给定初始点 x_0 , 需要近似的目标量为

$$u(0, x_0),$$

它对应原始正向时间方程中 $v(T, x_0)$ 的值. 在高维 PDE 数值算法文献中, Allen–Cahn 方程也常作为反应型非线性问题的基准算例^[1-2]. 该算例主要用于检验本文方法对反应型非线性项的处理能力.

3. 在统一框架下, 带违约风险的非线性 Black–Scholes 方程可对应

$$\mu(t, x) = \bar{\mu}x, \quad \sigma(t, x) = \bar{\sigma} \text{diag}(x),$$

以及

$$f(t, x, y, z) = -(1 - \delta)Q(y)y - Ry.$$

代入统一形式后得到

$$\frac{\partial u}{\partial t}(t, x) + \bar{\mu}x \cdot \nabla_x u(t, x) + \frac{\bar{\sigma}^2}{2} \sum_{i=1}^d x_i^2 \frac{\partial^2 u}{\partial x_i^2}(t, x) - (1 - \delta)Q(u(t, x))u(t, x) - Ru(t, x) = 0.$$

带违约风险的非线性 Black–Scholes 方程是金融衍生品定价中的一个重要扩展模型. 经典 Black–Scholes 方程假设市场不存在违约风险, 此时方程是线性的. 但在实际金融市场中, 合约发行方可能发生违约, 合约持有人只能获得部分价值补偿. 若违约强度依赖于当前合约价值, 则定价方程中会出现关于 u 的非线性项. 设 $x = (x_1, \dots, x_d)$ 表示 d 个标的资产价格, 在风险中性测度下资产价格可建模为几何布朗运动. 其中 R 为无风险利率, $\delta \in [0, 1)$ 表示违约后的回收比例, $Q(u)$ 表示与当前合约价值有关的违约强度. 在 Deep BSDE 的数值实验中, 非线性 Black–Scholes 方程也被用来展示金融定价问题中非线性项带来的高维求解困难^[1]. 该方程展示了金融定价模型中非线性项的一个典型来源: 市场风险因素或信用风险因素会使得原本线性的定价方程变为非线性.

4. 在统一框架下, Fisher–KPP 方程可取

$$\mu(t, x) = 0, \quad \sigma(t, x) = \sqrt{2}I_d,$$

以及

$$f(t, x, y, z) = y(1 - y).$$

代入统一形式后得到终端值问题

$$\frac{\partial u}{\partial t}(t, x) + \Delta u(t, x) + u(t, x)(1 - u(t, x)) = 0.$$

Fisher-KPP 方程是另一类经典的非线性反应-扩散方程, 常用于描述种群扩散和有利基因传播等问题. 其典型特点是反应项具有 Logistic 增长结构: 当解较小时近似线性增长, 当解接近饱和值时增长受到抑制. 因此, 该方程常被用来刻画带有扩散机制的传播前沿问题. 对应的正向时间形式为

$$\frac{\partial v}{\partial s}(s, x) = \Delta v(s, x) + v(s, x)(1 - v(s, x)).$$

通过同样的时间反向变换 $u(t, x) = v(T - t, x)$, 可以得到上面的终端值问题. 与 Allen-Cahn 方程类似, Fisher-KPP 方程也是反应-扩散型方程, 但其非线性项对应的是 Logistic 型增长, 而不是双势阱结构.

第三节 随机特征方法

随机特征法的基本思想是将两层神经网络中的内层参数随机生成并固定, 只求解输出层的线性系数. 与普通神经网络需要同时训练多层权重不同, 这种处理把函数逼近问题化为关于输出层参数的优化问题, 在平方损失下通常可以进一步写成最小二乘问题. 因此, 随机特征方法和极限学习机 (Extreme Learning Machine, ELM) 等线性化浅层网络方法都可以看作在表达能力和求解复杂度之间作出的折中^[17-19]. 从核方法角度看, 随机特征也可以理解为用随机 Monte Carlo 估计替代精确核计算, 其收敛和方差性质会影响实际近似效果^[20].

在常规低维 PDE 问题中, RFM 通常以配置点残差最小化的形式使用. 以边值问题

$$\mathcal{L}u(x) = f(x), \quad x \in \Omega, \quad \mathcal{B}u(x) = g(x), \quad x \in \partial\Omega$$

为例, 传统 FDM, FEM 和 SM 等方法通常需要先构造网格或选取确定性基函数, 再由离散方程得到有限维代数系统. RFM 则直接用一组固定的随机特征函数表示未知解, 例如

$$u_H(x) = \sum_{j=1}^H \alpha_j \phi_j(x), \quad \phi_j(x) = \sigma(a_j \cdot x + b_j),$$

其中 a_j, b_j 随机生成后保持不变, α_j 为待求的输出层系数. 在区域内部配置点上要求 $\mathcal{L}u_H - f$ 尽可能小, 在边界配置点上要求 $\mathcal{B}u_H - g$ 尽可能小, 就得到形如

$$\sum_{x_i \in C_I} \lambda_I |\mathcal{L}u_H(x_i) - f(x_i)|^2 + \sum_{x_j \in C_B} \lambda_B |\mathcal{B}u_H(x_j) - g(x_j)|^2$$

的损失函数. 这种随机特征表示, 配置点方法和边界条件惩罚可以被统一到同一个求解框架中, 从而在传统 PDE 数值方法和机器学习方法之间建立联系^[17]. 由于 PDE 残差和边界条件可以在同一个最小二乘目标中处理, 该方法不必为特征函数单独构造满足边界条件的形式, 在复杂区域上具有较好的使用灵活性. 在实际计算中, 还可以进一步引入分片单位分解, 多尺度表示和权重重标定等技巧, 用于改善局部尺度变化和不同残差项量级不一致带来的数值问题^[17].

上述低维 RFM 方法说明, 随机特征的优势主要来自两个方面: 一是用固定特征函数避免对内层参数进行非凸训练, 二是利用配置点或采样点把 PDE 求解转化为最小二乘型问题. 但在高维抛物型方程中, 若仍直接在整个时空区域内布置配置点, 采样覆盖和边界处理都会迅速变得困难. 另一方面, 本文关心的是给定初始点处的函数值 $u(0, x_0)$, 并不需要在整个高维区域上恢复完整的解函数. 因此, 本文不直接用随机特征近似 $u(t, x)$, 而是先将抛物型 PDE 转化为 BSDE, 再沿由正向随机过程生成的样本路径近似每个时间层上的梯度相关变量 Z_k . 在这一框架下, 随机特征的输入不再是全区域配置点, 而是时间层和样本路径上的状态点, 近似形式写为

$$Z_k^{(i)} \approx \alpha H_k^{(i)}.$$

这样既保留了 RFM 固定内层参数, 只求解输出层参数的结构, 又利用 BSDE 路径采样避免在高维空间中构造网格. 在线性方程中, 终端匹配可以化为线性最小二乘系统; 在半线性方程中, 由于终端残差关于参数不再线性, 本文进一步采用 LM 方法求解相应的非线性最小二乘问题. 近来的线性化网络实验也指出, 当特征宽度增加时, 相关最小二乘系统的条件数可能成为影响精度和可扩展性的主要因素之一. 因而后文数值实验也会同时观察特征宽度, 样本数和测试残差对结果的影响^[19].

第四节 深度倒向随机微分方程方法

深度倒向随机微分方程方法 (Deep Backward Stochastic Differential Equation method, Deep BSDE method) 是本文数值实验中采用的对比方法. 从半线性抛物型 PDE 的 BSDE 表示出发, 可以沿正向随机过程生成样本路径, 并用神经网络近似每个离散时间层上的梯度项 $\sigma^T(t_k, X_{t_k}) \nabla u(t_k, X_{t_k})$; 这正是 Deep BSDE 方法的基本思路^[1,21]. 初值 Y_0 与各时间层神经网络参数共同作为待优化变量, 通过最小化终端误差的平方损失来训练模型, 其中终端误差为

$$Y_N - g(X_N).$$

该方法的特点是把 PDE 的离散求解写成沿随机路径传播的优化问题. 其中, 正向过程 X_t 由给定的漂移项 μ 和扩散项 σ 生成, 倒向过程中的 Z_t 则由神经网络

给出. 由于整个算法只需要在样本路径上计算, 不需要在高维空间中构造网格, 因而可以用于传统网格方法难以直接处理的高维问题. 在非线性 Black–Scholes 方程, HJB 方程和 Allen–Cahn 方程上的数值实验表明, 这种框架能够在较高维度下给出有效的数值近似^[1]. 这与关于机器学习和计算数学关系的综述性讨论是一致的: 对高维 PDE 而言, 困难的核心逐渐从网格离散转向高维函数表示和随机优化. 这一点已有较为系统的总结^[12].

与本文采用的随机特征方法相比, Deep BSDE 方法需要优化的参数更多. 在常见实现中, 每个时间层都需要用一个神经网络近似梯度函数, 因此参数量会随时间剖分数和网络规模增加. 训练过程需要对全部网络参数进行反向传播, 对网络宽度, 学习率, 初始化和训练轮数等超参数也较为敏感. 因此, 本文仅将 Deep BSDE 作为对比方法, 用于检验随机特征方法在相同 BSDE 离散框架下的计算效率与初值误差表现.

第五节 本文主要工作与章节安排

本文围绕 BSDE 离散框架下的随机特征方法展开, 目标是在不构造高维空间网格的前提下, 用较低复杂度的参数求解过程近似高维线性与半线性抛物型方程的初值. 主要工作包括以下几个方面.

首先, 本文从统一的半线性抛物型方程出发, 通过 Itô 公式将终值问题转化为对应的 BSDE 形式, 并在离散时间层上构造样本路径. 在此基础上, 采用随机特征函数近似 BSDE 中的梯度相关变量 Z_k , 固定随机特征的内层参数, 只求解初值 y_0 和输出层参数 α . 这种处理保留了路径采样适合高维问题的特点, 同时避免了对多层神经网络全部参数进行训练.

其次, 对生成元关于 (y, z) 为线性的方程, 本文推导了离散终端值关于 y_0 和 α 的仿射表达式, 将终端匹配问题化为带岭正则化的线性最小二乘问题. 正则化只作用在随机特征输出层参数上, 不惩罚初值项, 以减少病态线性系统对参数规模的影响. 对半线性方程, 终端残差不再关于参数保持线性, 本文将问题写成非线性最小二乘形式, 并采用 LM 方法迭代求解. 为了提高迭代过程的可控性, 文中给出了终端残差关于参数的灵敏度递推, 用于构造 LM 子问题中的雅可比矩阵.

最后, 本文基于 C++ 和 libtorch 实现了上述求解流程, 并在线性 Black–Scholes 方程, 热方程以及半线性 HJB-LQ 方程, Allen–Cahn 方程上进行数值实验. 实验中同时报告训练路径上的终端残差和独立测试路径上的终端残差, 并考察样本数, 随机特征宽度, 维度和随机种子对结果的影响. 此外, Deep BSDE 方法仅作为对比方法出现在数值实验中, 用于观察本文随机特征方法在相同 BSDE 路径采样框架下的计算时间和初值误差表现.

后文结构安排如下. 第二章给出本文方法的具体构造, 包括 PDE 到 BSDE 的转化, 样本路径与随机特征的构造, 线性方程的最小二乘求解, 半线性方程的 LM 求解以及测试误差评估. 第三章给出四个高维算例的实验设置和数值结果, 并分析样本数, 特征宽度, 维度变化和随机种子稳定性等因素. 附录部分补充关键公式推导, 实验伪代码和参考值计算方法.

第二章 求解高维偏微分方程的随机特征方法

考虑方程 (1.2) 的终值问题, 即

$$\begin{aligned} \frac{\partial u}{\partial t}(t, x) + \frac{1}{2} \text{Tr}(\sigma \sigma^T(t, x) \nabla_x^2 u(t, x)) + \langle \mu(t, x), \nabla u(t, x) \rangle \\ + f(t, x, u(t, x), \sigma^T(t, x) \nabla u(t, x)) = 0, \\ (t, x) \in [0, T) \times \mathbb{R}^d, \\ u(T, x) = g(x), \quad x \in \mathbb{R}^d. \end{aligned} \quad (2.1)$$

为了应用 Itô 公式, 假设 $u \in C^{1,2}([0, T) \times \mathbb{R}^d)$, 并要求 μ, σ, f, g 满足足够的正则性与增长条件. 令状态过程满足

$$X_t = \xi + \int_0^t \mu(s, X_s) ds + \int_0^t \sigma(s, X_s) dW_s, \quad (2.2)$$

再定义

$$Y_t := u(t, X_t), \quad Z_t := \sigma^T(t, X_t) \nabla u(t, X_t). \quad (2.3)$$

则原方程可写成 BSDE

$$dY_t = -f(t, X_t, Y_t, Z_t) dt + Z_t^T dW_t, \quad (2.4)$$

并满足终端条件 $Y_T = g(X_T)$. 这种以 BSDE 表示抛物型 PDE 并在样本路径上求解的思路, 也是 Deep BSDE 等高维 PDE 算法的基础^[1-2]. 从 (2.1) 到 (2.4) 的推导见附录 A 第一节.

取时间剖分 $0 = t_0 < t_1 < \dots < t_N = T$, 其中 $\Delta t = T/N$, 并记 $\Delta W_k = W_{t_{k+1}} - W_{t_k}$. 对 (2.4) 采用 Euler 离散后, 得到

$$Y_{k+1} = Y_k - \Delta t f(t_k, X_k, Y_k, Z_k) + \langle Z_k, \Delta W_k \rangle. \quad (2.5)$$

因而问题转化为: 在离散路径上同时近似初值 Y_0 和各时间层的梯度项 Z_k , 使终端值与 $g(X_T)$ 尽可能一致.

第一节 整体思路

本文的实验流程可以概括为四个阶段:

1. 根据给定方程生成离散样本路径;
2. 在所有时间层和路径点上构造随机特征, 用于近似 Z_k ;
3. 根据方程类型选择线性最小二乘或非线性 Levenberg-Marquardt 方法 (Levenberg-Marquardt method, LM method) 求解参数;
4. 在独立测试路径上重新推进终端值, 计算测试残差.

上述四步中, 前两步构成公共的数据准备阶段, 对线性和半线性问题都相同; 第三步则是两类方程的主要差异所在. 在线性情形下, 末端匹配可以直接化为一个最小二乘系统; 在半线性情形下, 则需要通过迭代方式不断修正参数. 本文实验的统一框架如下算法 2.1.

算法 2.1 统一实验框架

Data: 方程参数, 时间剖分数 N , 样本数 S , 随机特征宽度 H

Result: 训练得到的 y_0, α , 以及训练与测试误差

- 1 生成训练样本路径 $\{X_k^{(i)}, \Delta W_k^{(i)}\}$;
 - 2 **(Parallel) for** $k = 0, \dots, N - 1$ **do**
 - 3 构造第 k 个时间层上的随机特征 $\{H_k^{(i)}\}_{i=1}^S$;
 - 4 **end**
 - 5 **if** 方程为线性 **then**
 - 6 调用算法 B.1 求解 y_0 和 α ;
 - 7 **else**
 - 8 调用算法 B.2 求解 y_0 和 α ;
 - 9 **end**
 - 10 在训练路径上计算终端残差 RMSE;
 - 11 在独立测试路径上调用算法 B.3 计算测试 RMSE;
 - 12 返回求解参数与误差指标;
-

第二节 样本路径与随机特征的构造

设终值问题已经被离散为 N 个时间步, 样本数为 S . 对每个实验算例, 首先按照对应随机微分方程生成 S 条独立样本路径

$$\{X_k^{(i)}\}_{k=0}^N, \quad i = 1, 2, \dots, S,$$

以及与之对应的布朗增量

$$\{\Delta W_k^{(i)}\}_{k=0}^{N-1}.$$

这些路径同时承担两个作用: 一方面它们给出离散 BSDE 的推进轨道, 另一方面它们也是随机特征映射的输入样本.

在得到路径后, 对每个时间层和路径点计算随机特征

$$H_k^{(i)} = \tanh(Ax_k^{(i)} + bt_k + c),$$

其中 A, b, c 为固定随机参数. 本文始终只训练输出层参数 α , 并用

$$Z_k^{(i)} \approx \alpha H_k^{(i)}$$

作为梯度项的统一近似. 于是原问题的求解就转化为对 y_0 和 α 的求解.

第三节 线性方程的求解

对线性方程, 由第二章的推导可知, 每条路径的终端值都可以写成

$$y_N^{(i)} = \beta_i y_0 + \langle \Gamma_i, \alpha \rangle + \eta_i.$$

因而所有路径拼接后, 就得到一个关于参数向量 $(y_0, \text{vec}(\alpha))$ 的线性最小二乘问题.

从实验流程的角度看, 线性情形可分为以下几步:

1. 生成样本路径并计算随机特征;
2. 根据离散递推式计算每条路径的系数 $\beta_i, \Gamma_i, \eta_i$;
3. 将所有路径系数拼接成统一的设计矩阵;
4. 求解最小二乘问题得到 y_0 与 α ;
5. 用求得的参数回代, 计算训练样本上的终端残差.

线性方程中参数求解的具体流程见附录四 算法 B.1.

线性流程的特点在于参数只需求解一次, 不需要额外的外层迭代. 因此实验中线性部分主要用于观察样本数, 特征宽度以及维度变化对最小二乘近似效果的影响.

先考虑线性生成元

$$f(t, \mathbf{x}, y, \mathbf{z}) = l(t, \mathbf{x})y + \langle \mathbf{m}(t, \mathbf{x}), \mathbf{z} \rangle + n(t, \mathbf{x}). \quad (2.6)$$

其中 $\langle \mathbf{m}, \mathbf{z} \rangle$ 可以并入漂移项处理, 因此后文只考虑 $\mathbf{m} \equiv \mathbf{0}$ 的情形.

对每条样本路径 i , 用随机特征映射逼近梯度项

$$\mathbf{Z}_k^{(i)} \approx \alpha \mathbf{H}_k^{(i)}, \quad \mathbf{H}_k^{(i)} = \tanh(\mathbf{A} \mathbf{x}_k^{(i)} + b t_k + c), \quad (2.7)$$

其中 $\mathbf{A}, b, c$ 为固定随机参数, α 为待训练的输出层参数. 代入 (2.5) 后得到离散迭代

$$y_{k+1}^{(i)} = (1 - \Delta t l(t_k, \mathbf{x}_k^{(i)})) y_k^{(i)} + (\Delta \mathbf{W}_k^{(i)})^T \alpha \mathbf{H}_k^{(i)} - \Delta t n(t_k, \mathbf{x}_k^{(i)}). \quad (2.8)$$

由于上式关于 y_0 和 α 是线性的, 终端值 $y_N^{(i)}$ 也必然是它们的仿射函数. 将递推完全展开后, 可写为

$$y_N^{(i)} = \beta_i y_0 + \langle \Gamma_i, \alpha \rangle + \eta_i, \quad (2.9)$$

其中 $\beta_i, \Gamma_i, \eta_i$ 由路径 $\{\mathbf{x}_k^{(i)}, \Delta \mathbf{W}_k^{(i)}\}$ 唯一确定, 具体表达式分别见附录 A 第二节的 (A.12), (A.13) 和 (A.14).

将所有路径的终端值拼接后, 便得到线性最小二乘问题. 记

$$\vartheta = \begin{bmatrix} y_0 \\ \text{vec}(\alpha) \end{bmatrix}, \quad \Phi_{i,\cdot} = \begin{bmatrix} \beta_i & \text{vec}(\Gamma_i)^T \end{bmatrix}, \quad B_i = g(\mathbf{x}_N^{(i)}) - \eta_i. \quad (2.10)$$

则线性方程最终求解

$$\min_{\vartheta} \|\Phi\vartheta - B\|_2^2 + \lambda_r \|R\vartheta\|_2^2, \quad (2.11)$$

其中 $\lambda_r > 0$ 为岭正则化 (ridge regularization) 参数,

$$R = \begin{bmatrix} 0 & 0 \\ 0 & I_{dH} \end{bmatrix}. \quad (2.12)$$

也就是说, 正则化只作用在随机特征输出层参数 α 上, 不惩罚初值 y_0 . 这样做可以避免把 y_0 人为向零偏移, 同时抑制病态线性系统给出的过大 α .

实际计算时, 设未知量个数为 $p = 1 + dH$. 当样本数 $S \geq p$ 时, 直接求解原始岭正则化系统

$$(\Phi^T \Phi + \lambda_r R^T R) \vartheta = \Phi^T B. \quad (2.13)$$

当 $S < p$ 时, 使用对偶形式降低线性系统规模. 由于当前实现的对偶形式主要用于欠定情形, 其正则化写为

$$\vartheta = \Phi^T (\Phi \Phi^T + \lambda_r I_S)^{-1} B. \quad (2.14)$$

这种情形下系统欠定, 并不能得到较好的结果. 此处仅作为程序鲁棒性的扩展, 不在这种情形下实验和分析.

线性情形的优势在于不需要迭代更新随机特征参数, 每次实验只需求解一个最小二乘系统即可.

第四节 半线性方程的求解

对半线性方程, 终端值关于参数不再保持线性, 因而不能直接写成单个最小二乘系统. 这时需要把参数记为

$$\theta = (y_0, \text{vec}(\alpha)),$$

并通过最小化终端残差平方和来求解

$$\min_{\theta} \frac{1}{2} \|r(\theta)\|_2^2.$$

本文实验中对半线性方程采用 LM 方法. 每次迭代都围绕当前参数执行以下流程:

1. 根据当前参数构造 $Z_k^{(i)} = \alpha H_k^{(i)}$;
2. 沿所有样本路径前向推进离散 BSDE, 得到终端残差;
3. 计算残差关于参数的雅可比矩阵;
4. 求解阻尼线性子问题, 得到参数增量;
5. 比较更新前后的目标函数值, 决定是否接受该步并调整阻尼参数.

这一步骤会持续进行, 直到终端残差足够小, 参数更新足够小, 或达到最大迭代次数. 因而半线性实验的核心并不在于单次前向传播本身, 而在于“前向残差评估 + 雅可比构造 + 阻尼更新”这一循环是否稳定. 半线性方程的 LM 迭代流程见附录四 算法 B.2.

对一般半线性生成元 $f(t, \mathbf{x}, y, \mathbf{z})$, 仍采用相同的随机特征近似

$$\mathbf{Z}_k^{(i)} \approx \alpha \mathbf{H}_k^{(i)}. \quad (2.15)$$

此时离散迭代式为

$$y_{k+1}^{(i)} = y_k^{(i)} - f(t_k, x_k^{(i)}, y_k^{(i)}, \alpha \mathbf{H}_k^{(i)}) \Delta t + (\Delta W_k^{(i)})^T \alpha \mathbf{H}_k^{(i)}. \quad (2.16)$$

由于生成元 f 关于 y 或 z 不再保持线性, 终端值 $y_N^{(i)}$ 不能再写成关于参数的线性函数, 因此末端匹配需要改写为半线性最小二乘问题.

令参数向量为

$$\theta = \begin{bmatrix} y_0 \\ \text{vec}(\alpha) \end{bmatrix}, \quad (2.17)$$

对第 i 条路径定义终端残差

$$r_i(\theta) = y_N^{(i)}(\theta) - g(x_N^{(i)}), \quad (2.18)$$

并记

$$r(\theta) = \begin{bmatrix} r_1(\theta) \\ \vdots \\ r_S(\theta) \end{bmatrix}. \quad (2.19)$$

最终求解问题为

$$\min_{\theta} \frac{1}{2} \|r(\theta)\|_2^2. \quad (2.20)$$

本文采用 LM 方法求解 (2.20). 在第 m 次迭代中, 将残差在当前参数 $\theta^{(m)}$ 处线性化:

$$r(\theta^{(m)} + \delta) \approx r(\theta^{(m)}) + J(\theta^{(m)})\delta, \quad (2.21)$$

其中

$$J(\theta^{(m)}) = \frac{\partial r}{\partial \theta}(\theta^{(m)}) \quad (2.22)$$

为雅可比矩阵. 于是每步需要求解阻尼线性系统

$$(J^T J + \lambda I) \delta^{(m)} = -J^T r, \quad \lambda > 0, \quad (2.23)$$

并更新

$$\theta^{(m+1)} = \theta^{(m)} + \delta^{(m)}. \quad (2.24)$$

若目标函数下降, 则接受该步并减小阻尼参数; 否则拒绝该步并增大阻尼参数.

为了构造 (2.23), 需要计算终端残差对参数的灵敏度. 对第 i 条路径定义

$$S_k^{(i)} = \frac{\partial y_k^{(i)}}{\partial \theta}, \quad (2.25)$$

则有初值

$$S_0^{(i)} = \begin{bmatrix} 1 & 0 & \dots & 0 \end{bmatrix}, \quad (2.26)$$

并满足递推

$$S_{k+1}^{(i)} = \left(1 - \Delta t f_{y,k}^{(i)}\right) S_k^{(i)} + \left((\Delta W_k^{(i)})^T - \Delta t f_{z,k}^{(i)}\right) \begin{bmatrix} 0_{d \times 1} & I_d \otimes (H_k^{(i)})^T \end{bmatrix}, \quad (2.27)$$

其中

$$f_{y,k}^{(i)} = \frac{\partial f}{\partial y}(t_k, x_k^{(i)}, y_k^{(i)}, \alpha H_k^{(i)}), \quad f_{z,k}^{(i)} = \frac{\partial f}{\partial z}(t_k, x_k^{(i)}, y_k^{(i)}, \alpha H_k^{(i)}). \quad (2.28)$$

因而第 i 条路径对应的雅可比行向量就是

$$J_i(\theta) = S_N^{(i)} = \left(\prod_{j=0}^{N-1} \Phi_j^{(i)}\right) S_0^{(i)} + \sum_{k=0}^{N-1} \left(\prod_{j=k+1}^{N-1} \Phi_j^{(i)}\right) B_k^{(i)}. \quad (2.29)$$

上述灵敏度递推及其由来见附录 A 第三节.

这与线性方程中系数矩阵的推导形式类似, 区别在于此处 $\Phi_k^{(i)}$ 和 $B_k^{(i)}$ 依赖当前迭代步中的 $y_k^{(i)}$ 和 α , 因而需要在每一次迭代中重新计算. 但随机特征结构使得待优化参数始终只出现在输出层, 从而降低了优化问题的规模.

第五节 测试阶段与误差评估

为了避免只在固定训练路径上评价模型, 本文在训练完成后还会重新采样独立测试路径. 对测试路径仍然使用相同的随机特征参数和输出层参数, 并按照与训练阶段一致的离散格式推进终端值. 由此得到的测试误差用均方根误差 (Root Mean Square Error, RMSE) 表示, 记为

$$\text{RMSE}_{\text{test}} = \sqrt{\frac{1}{S} \sum_{i=1}^S (Y_N^{(i)} - g(X_N^{(i)}))^2}.$$

测试误差的具体评估流程见附录四 算法 B.3.

若训练误差下降而测试误差没有同步下降, 则说明随机特征近似可能更偏向于拟合当前训练路径; 反之, 若两者同时下降, 则说明该参数设置具有更稳定的泛化效果. 因而训练 RMSE 与测试 RMSE 的对比是本实验章节中的核心观察指标之一.

第三章 数值实验

本章通过两个线性方程和两个半线性方程测试前文方法. 线性方程选取 Black–Scholes 方程和热方程, 主要观察线性最小二乘求解器的精度; 半线性方程选取线性二次型 HJB 方程和 Allen–Cahn 方程, 用于观察 LM 方法在高维 BSDE 离散系统中的效果. 这些算例与高维 PDE 学习型算法文献中常用的测试问题一致, 便于和已有 Deep BSDE 及随机特征方法相关结果进行对照^[1,17].

第一节 实验设置

每个算例都先生成样本路径 $\{X_k^{(i)}, \Delta W_k^{(i)}\}_{k=0}^{N-1}$, 然后用随机特征函数近似各时间层的 Z_k . 对线性方程, 终端值 Y_N 关于 y_0 和输出层参数 α 保持线性, 所以可以直接求解带岭正则化 (ridge regularization) 的最小二乘问题. 对半线性方程, 终端残差关于参数不再是线性的, 本文采用 LM 方法求解相应的非线性最小二乘问题.

本文实现中, 随机特征函数取为

$$H_k^{(i)} = \tanh\left(x_k^{(i)} A + t_k b + c\right), \quad (3.1)$$

其中 $A \in \mathbb{R}^{d \times H}$, $b, c \in \mathbb{R}^{1 \times H}$ 为生成后固定的内层参数. 在程序实现中, 各分量按

$$A_{mh} \sim \mathcal{N}\left(0, \frac{1}{d}\right), \quad b_h \sim \mathcal{N}(0, 1), \quad c_h \sim \mathcal{N}(0, 1), \quad (3.2)$$

独立采样, 并由同一个实验随机种子派生出不同子种子, 分别用于生成 A , b 和 c . 采样后这些内层参数在训练和测试阶段都保持不变, 只求解输出层系数 α . 这种设置与随机特征方法固定内层参数的思想一致. 其中 A 的标准差取为 $1/\sqrt{d}$, 是为了使 $x_k^{(i)} A$ 的量级在维度增大时仍保持相对稳定: 若 x 的各分量量级相近, 则每个特征输入中的空间部分方差约与 $\|x\|^2/d$ 同阶, 不会因为 d 增大而直接放大. 这样可以减少 $\tanh$ 输入过大而落入饱和区间的情况. 时间权重 b 和偏置 c 采用标准正态分布, 用于在不同时间层和不同随机平移下生成多样的特征函数. 需要说明的是, 这种采样方式不是唯一选择, 特征宽度和随机种子仍会影响最终结果, 因而后文还会单独考察 H 和随机种子对数值结果的影响.

本章统一报告训练样本上的终端残差均方根误差 (Root Mean Square Error, RMSE) 和独立测试样本上的终端残差 RMSE. 前者反映模型在固定样本路径上的拟合程度, 后者用于观察参数是否只适应了训练路径. 除特别说明外, 标准配置见表 3.1.

本章还大量使用了方程初值的参考真实值, 其具体计算方法和误差或置信区

表 3.1 主要实验配置

方程	类型	d	T	N	S	H	λ_0	求解器
Black–Scholes	线性	100	1.0	20	16384	50	0.010	线性最小二乘
Heat	线性	100	1.0	20	16384	50	0.010	线性最小二乘
HJB-LQ	半线性	100	1.0	20	16384	50	0.010	LM
Allen–Cahn	半线性	100	0.3	20	16384	50	0.010	LM

间均见附录 B.

实验在 5060ti 16G GPU 上完成, 使用 C++/libtorch, 数据类型为 float32. 运行时间包括样本路径生成, 随机特征构造, 线性系统求解和测试误差计算.

第二节 线性方程

本节考虑生成元关于 (y, z) 为线性的方程. 对这类问题, 离散 BSDE 的终端值可以写成

$$Y_N^{(i)} = A_{i,0}y_0 + \sum_{j=1}^{dH} A_{i,j}\theta_j + b_i, \quad \theta = \text{vec}(\alpha). \quad (3.3)$$

因此, 由终端匹配条件 $Y_N^{(i)} \approx g(X_N^{(i)})$ 可以直接得到一个线性最小二乘问题, 不需要再做外层非线性迭代.

Black–Scholes 算例采用多资产算术平均看涨期权. Black–Scholes 模型最初针对欧式期权定价提出, 而多资产形式则常用于检验高维金融定价算法的可扩展性^[14,16]. 为避免与统一框架中的扩散系数函数 $\sigma(t, x)$ 混淆, 这里记常数波动率为 σ_{BS} . 统一框架下取

$$\mu(t, x) = rx, \quad \sigma(t, x) = \sigma_{\text{BS}} \text{diag}(x), \quad f(t, x, y, z) = -ry. \quad (3.4)$$

具体方程为

$$\frac{\partial u}{\partial t}(t, x) + \frac{1}{2}\sigma_{\text{BS}}^2 \sum_{m=1}^d x_m^2 \frac{\partial^2 u}{\partial x_m^2}(t, x) + r \sum_{m=1}^d x_m \frac{\partial u}{\partial x_m}(t, x) - ru(t, x) = 0, \quad (3.5)$$

终端条件为

$$u(T, x) = g(x) = \max\left(\frac{1}{d} \sum_{m=1}^d x_m - K, 0\right). \quad (3.6)$$

本文实验取

$$d = 100, \quad T = 1, \quad x_0 = (1, \dots, 1), \quad r = 0.05, \quad \sigma_{\text{BS}} = 0.2, \quad K = 1.$$

对应状态过程满足

$$X_{k+1}^m = X_k^m \exp\left((r - \frac{1}{2}\sigma_{BS}^2)\Delta t + \sigma_{BS}\Delta W_k^m\right), \quad m = 1, \dots, d, \quad (3.7)$$

该算例没有类似一维 Black-Scholes 公式的简单闭式解, 本文采用附录 B 第三节中的控制变量 Monte Carlo 参考值.

Heat 算例在统一框架下取

$$\mu(t, x) = 0, \quad \sigma(t, x) = I_d, \quad f(t, x, y, z) = 0. \quad (3.8)$$

具体方程为

$$\frac{\partial u}{\partial t}(t, x) + \frac{1}{2}\Delta u(t, x) = 0, \quad u(T, x) = g(x). \quad (3.9)$$

本文实验取

$$d = 100, \quad T = 1, \quad x_0 = 0, \quad g(x) = \exp\left(-\frac{\|x\|^2}{a}\right), \quad a = d. \quad (3.10)$$

状态过程为 $X_t = x_0 + W_t$. 由 Feynman-Kac 公式, 该算例具有显式解

$$u(t, x) = \left(\frac{a}{a + 2(T - t)}\right)^{d/2} \exp\left(-\frac{\|x\|^2}{a + 2(T - t)}\right). \quad (3.11)$$

因而在上述参数下 $Y_0 = (d/(d + 2))^{d/2}$.

表 3.2 线性方程主要结果

方程	y_0	参考值	相对误差	训练 RMSE	测试 RMSE
Black-Scholes	0.048726	0.048813	0.178%	5.2851×10^{-2}	7.4640×10^{-2}
Heat	0.372581	0.371528	0.283%	3.5968×10^{-2}	5.1693×10^{-2}

表 3.2 给出了两个线性算例的结果. 可以看到, 两个算例得到的初值都与独立参考值接近. Black-Scholes 的相对误差约为 0.18%, Heat 方程的相对误差约为 0.28%.

第三节 半线性方程

HJB-LQ 方程的非线性来自梯度项, Allen-Cahn 方程的非线性来自反应项. 这两个算例分别对应 z 型非线性和 y 型非线性. 这两个算例也都出现在 Deep BSDE 方法的高维数值实验中, 因而适合用于观察本文随机特征方法在半线性问题上的表现^[1].

HJB-LQ 方程在统一框架下取

$$\mu(t, x) = 0, \quad \sigma(t, x) = \sqrt{2}I_d, \quad f(t, x, y, z) = -\frac{1}{2}\|z\|^2. \quad (3.12)$$

具体方程为

$$\frac{\partial u}{\partial t}(t, x) + \Delta u(t, x) - \|\nabla_x u(t, x)\|^2 = 0, \quad u(T, x) = g(x). \quad (3.13)$$

本文实验取

$$d = 100, \quad T = 1, \quad x_0 = 0, \quad g(x) = \log\left(\frac{1 + \|x\|^2}{2}\right). \quad (3.14)$$

状态过程为 $X_t = \sqrt{2}W_t$. 对该终端条件, 指数变换给出参考表达式

$$u(t, x) = -\log \mathbb{E} \left[\exp(-g(x + \sqrt{2}W_{T-t})) \right]. \quad (3.15)$$

特别地,

$$Y_0 = -\log \mathbb{E} \left[\frac{2}{1 + 2T\chi_d^2} \right], \quad (3.16)$$

数值参考值由附录 B 第二节中的一维积分计算.

Allen–Cahn 方程在统一框架下取

$$\mu(t, x) = 0, \quad \sigma(t, x) = \sqrt{2}I_d, \quad f(t, x, y, z) = y - y^3. \quad (3.17)$$

具体方程为

$$\frac{\partial u}{\partial t}(t, x) + \Delta u(t, x) + u(t, x) - u^3(t, x) = 0, \quad u(T, x) = g(x). \quad (3.18)$$

本文实验取

$$d = 100, \quad T = 0.3, \quad x_0 = 0, \quad g(x) = \frac{1}{2 + 0.4\|x\|^2}. \quad (3.19)$$

状态过程为 $X_t = \sqrt{2}W_t$. 该方程没有简单闭式解析解, 本文采用附录 B 第四节中的径向有限差分参考值.

表 3.3 半线性方程主要结果

方程	y_0	参考值	相对误差	训练 RMSE	测试 RMSE
HJB-LQ	4.590000	4.590162	0.0035%	9.9006×10^{-2}	1.4430×10^{-1}
Allen–Cahn	0.053131	0.052785	0.656%	3.5241×10^{-3}	5.1108×10^{-3}

表 3.3 表明, LM 求解后的两个半线性算例也能给出较准确的初值. HJB-LQ 的相对误差约为 0.0035%, Allen–Cahn 的相对误差约为 0.66%.

图 3.1 给出了半线性求解器的残差下降过程. 横轴为 LM 接受步编号, 纵轴为训练终端残差 RMSE.

表 3.4 半线性方程的 LM 迭代结果

方程	迭代步数	接受步数	最终 λ	最终步长范数	时间 / ms
HJB-LQ	4	3	1.00×10^4	2.82×10^{-7}	3377.930
Allen-Cahn	4	3	1.00×10^6	5.59×10^{-10}	3148.710

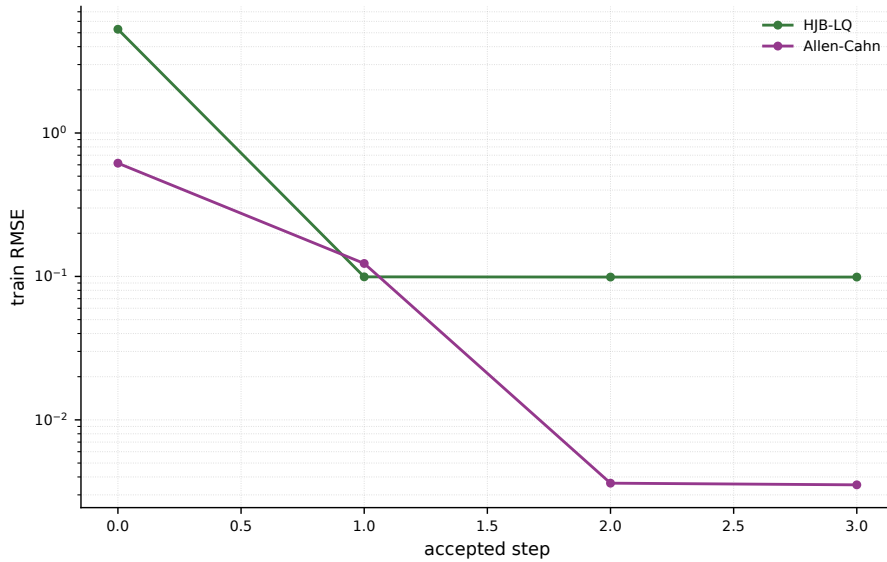


图 3.1 半线性方程的 LM 迭代残差下降

第四节 参数敏感度分析

一、样本数的影响

样本数会影响训练路径对真实分布的代表性. 表 3.5 固定 $d = 100$, $H = 50$, 只改变训练样本数 S .

表 3.5 样本数变化结果

方程	S	y_0	相对误差	训练 RMSE	测试 RMSE	测试/训练
Heat	4096	0.376310	1.287%	1.6712×10^{-4}	1.4144×10^{-1}	846.37
Heat	8192	0.373278	0.471%	2.7041×10^{-2}	6.9560×10^{-2}	2.57
Heat	16384	0.372581	0.283%	3.5968×10^{-2}	5.1693×10^{-2}	1.44
Black-Scholes	8192	0.048520	0.599%	8.3590×10^{-3}	1.9868×10^{-2}	2.38
Black-Scholes	16384	0.048726	0.178%	5.2851×10^{-2}	7.4640×10^{-2}	1.41
HJB-LQ	4096	4.423910	3.622%	1.3631×10^{-6}	4.5946×10^{-1}	3.37×10^5
HJB-LQ	8192	4.578700	0.250%	7.6157×10^{-2}	1.9626×10^{-1}	2.58
HJB-LQ	16384	4.590000	0.0035%	9.9006×10^{-2}	1.4430×10^{-1}	1.46
Allen-Cahn	8192	0.053268	0.916%	2.7531×10^{-3}	6.8620×10^{-3}	2.49
Allen-Cahn	16384	0.053131	0.656%	3.5241×10^{-3}	5.1108×10^{-3}	1.45

当样本数较小时, 训练残差本身不一定可靠. 随机种子决定了内层随机特征, 也会影响有效特征的数量; 在样本较少时, 某些随机特征组合可能很好地拟合训

练路径,但在独立测试路径上误差仍然较大.因此表中加入测试 RMSE 与训练 RMSE 的比值,作为观察泛化情况的辅助指标.例如 HJB-LQ 在 $S = 4096$ 时训练 RMSE 几乎为零,但测试/训练比值达到 3.37×10^5 ,初值相对误差也偏大.因此后续分析主要参考独立测试残差和初值相对误差.图 3.2 给出了测试/训练比值和初值相对误差的对比.

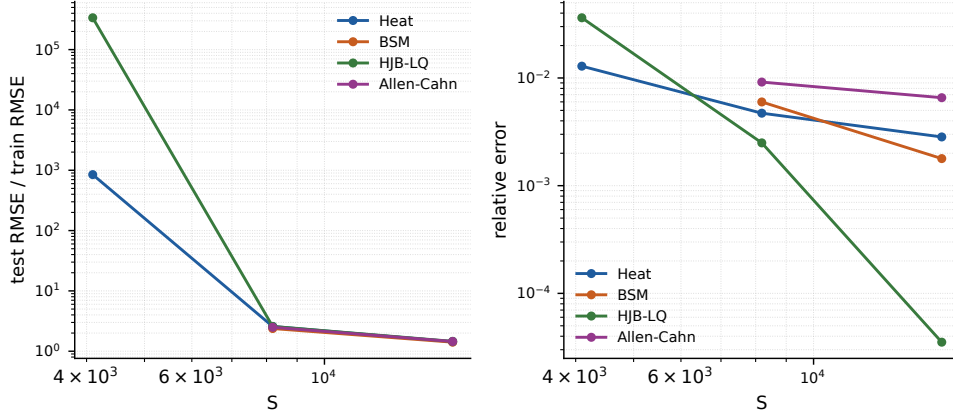


图 3.2 样本数对泛化比值和初值相对误差的影响

二、随机特征宽度的影响

随机特征宽度 H 决定每个时间层上可训练输出参数的数量. 本节固定 $d = 100$, $S = 16384$, 只改变 H . 对 HJB-LQ 方程, 增大 H 后初值相对误差和训练残差都有改善; 对 Heat, Black-Scholes 和 Allen-Cahn, 当 H 已达到当前规模后, 继续增大 H 并不能稳定带来更高精度. 这说明随机特征宽度并不是唯一的限制因素; 样本路径数量, 随机特征的有效线性无关程度以及最小二乘系统的条件数都会共同影响结果. 图 3.3 展示了测试残差和初值相对误差随 H 变化的情况.

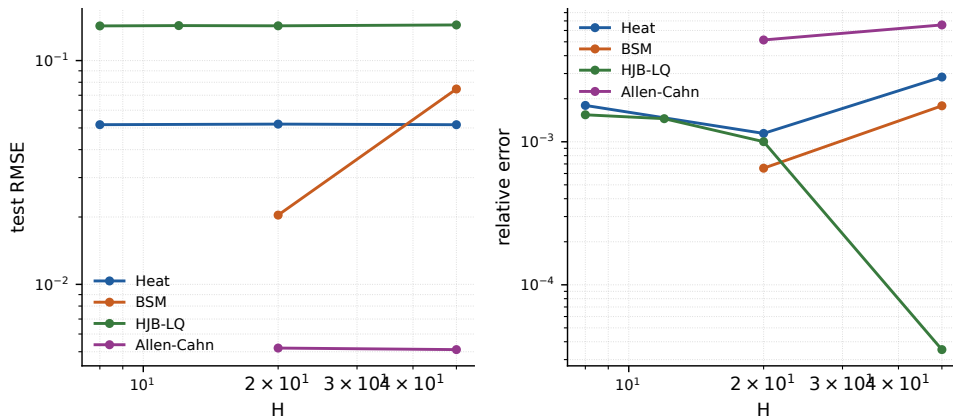


图 3.3 随机特征宽度对测试残差和初值相对误差的影响

三、维度与运行时间

为了观察维度变化对运行时间的影响, 本节固定 $H = 50$, $S = 16384$, 对 $d = 20, 30, 50, 100, 200, 300$ 分别运行三个随机种子并取平均. 由于总运行时间包含初始化, 公用数据构造和求解等多个部分, 直接用总时间拟合维度阶数并不合适. 因此对相邻两组维度 d_{j-1} 和 d_j , 定义局部阶

$$p_j = \frac{\log(t_j/t_{j-1})}{\log(d_j/d_{j-1})}, \quad (3.20)$$

其中 t_j 表示该维度下三次运行的平均时间. 表 3.6 列出了不同维度下的平均时间和相邻维度的局部阶.

表 3.6 维度变化的平均结果

方程	d	时间 / ms	p_j	方程	d	时间 / ms	p_j
Heat	20	204.4 ± 3.5	--	Black-Scholes	20	209.5 ± 5.1	--
Heat	30	216.0 ± 3.6	0.14	Black-Scholes	30	219.1 ± 2.0	0.11
Heat	50	238.8 ± 2.7	0.20	Black-Scholes	50	242.7 ± 4.2	0.20
Heat	100	314.7 ± 3.8	0.40	Black-Scholes	100	325.3 ± 4.5	0.42
Heat	200	568.8 ± 1.3	0.85	Black-Scholes	200	599.1 ± 9.9	0.88
Heat	300	1031.7 ± 4.8	1.47	Black-Scholes	300	1057.3 ± 0.3	1.40
HJB-LQ	20	435.5 ± 69.2	--	Allen-Cahn	20	444.7 ± 65.6	--
HJB-LQ	30	680.6 ± 92.5	1.10	Allen-Cahn	30	648.0 ± 112.1	0.93
HJB-LQ	50	1053.4 ± 176.5	0.85	Allen-Cahn	50	856.4 ± 90.0	0.55
HJB-LQ	100	3273.2 ± 263.6	1.64	Allen-Cahn	100	2780.4 ± 649.2	1.70
HJB-LQ	200	5616.8 ± 3150.0	0.78	Allen-Cahn	200	6472.6 ± 3419.0	1.22
HJB-LQ	300	27300.8 ± 4323.0	3.90	Allen-Cahn	300	40998.0 ± 7671.0	4.55

从表 3.6 可以看到, 线性方程在低维到中等维度上的局部阶较小, 到 $d = 200, 300$ 时局部阶逐渐增大; 半线性方程的时间增长更快, 尤其在 $d = 300$ 时受到非线性迭代和线性代数规模的共同影响. 这里的高维端局部阶升高, 不宜直接解释为算法本身的理论复杂度. 一方面, 维度增大后, 参数矩阵和样本路径占用的显存增加较多, GPU 并行度可能受到显存带宽, 缓存命中率和临时张量分配的限制; 另一方面, 半线性方程每次 LM 试探步都要重新传播路径并构造灵敏度, 拒绝步和重试次数会直接放大运行时间. 此外, 当问题接近实验设备的显存上限时, 内存碎片, kernel 调度开销, 以及矩阵求解例程在大尺寸矩阵上的工作区需求都可能造成额外开销. 本实验中 $d = 300$, $H = 50$, $S = 16384$ 已接近设备可运行上限, 因而这个端点更适合作为硬件压力测试, 而不是复杂度阶数的稳定估计点. 图 3.4 左图在 log-log 坐标下展示了平均运行时间随维度变化的趋势. 除运行时间外, 还需要检查高维下的初值精度. 图 3.4 右图给出了不同维度, 不同随机种子下的初值相对误差. 图中的误差大多仍然较小, 说明方法在较高维度下仍能给出有效初值.

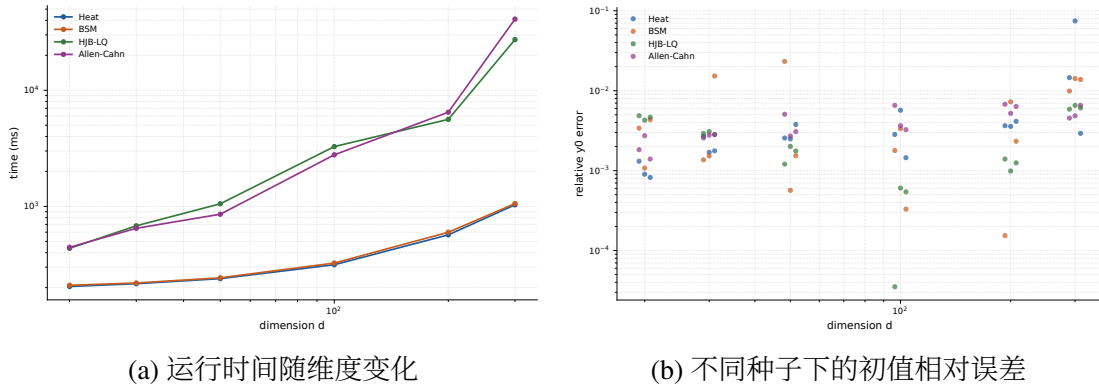


图 3.4 维度变化下的运行时间和初值相对误差

四、随机种子稳定性

随机特征内层参数和样本路径都由随机数生成, 因此需要检查结果是否依赖某个特定种子. 表 3.7 使用标准配置下的三个种子, 报告 y_0 均值, 样本标准差和平均测试残差.

表 3.7 随机种子稳定性结果

方程	y_0 均值	y_0 标准差	均值相对误差	测试 RMSE 均值
Heat	0.372764	8.04×10^{-4}	0.333%	5.1965×10^{-2}
Black-Scholes	0.048843	1.26×10^{-4}	0.063%	3.7928×10^{-2}
HJB-LQ	4.591863	1.62×10^{-3}	0.037%	1.4361×10^{-1}
Allen-Cahn	0.053021	9.54×10^{-5}	0.449%	5.0785×10^{-3}

四个方程的 y_0 波动都小于主要数值量级. 图 3.5 给出每个随机种子下的初值相对误差.

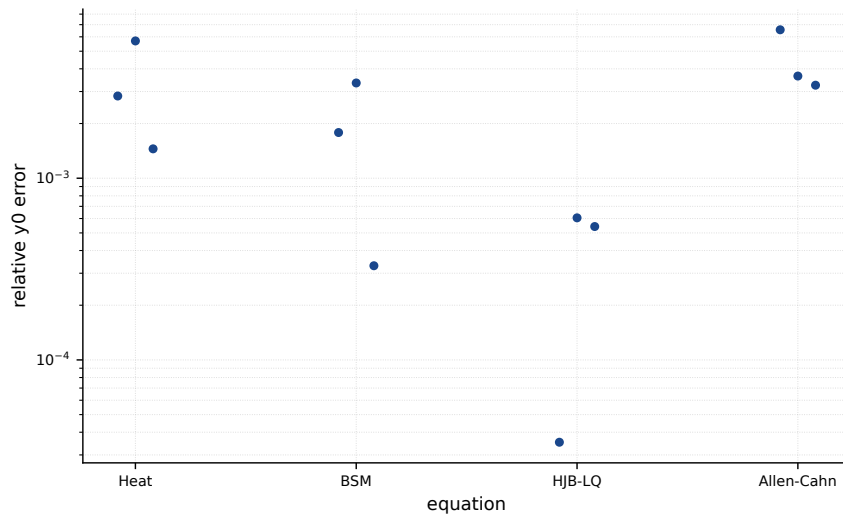


图 3.5 不同随机种子下的初值相对误差

第五节 Deep BSDE 对比实验

第四节介绍的 Deep BSDE 对比方法, 属于一类基于神经网络求解高维抛物型方程的代表性做法^[1]. 该方法同样先将方程转化为 BSDE, 再拟合其中的梯度项. 与本文方法不同, Deep BSDE 使用标准多层感知机 (Multilayer Perceptron, MLP) 近似梯度项, 需要通过迭代训练更新网络参数. 为了进行比较, 本文实现了文献中的算法, 并在相同方程参数和计算环境下进行实验. 本文在 HJB-LQ 和 Allen-Cahn 两个半线性方程上记录了 Deep BSDE 的训练过程. 由于 Deep BSDE 的训练损失和本文 RFM 的终端残差并不是同一个优化目标, 因此这里不直接比较损失数值, 只比较初值相对误差随时间的变化.

表 3.8 本文方法与 Deep BSDE 对比方法在半线性方程上的结果

方程	方法	时间 / s	y_0	参考值	相对误差
HJB-LQ	本文方法 (RFM)	3.378	4.590000	4.590162	0.0035%
HJB-LQ	对比方法 (Deep BSDE)	15.221	4.586627	4.590162	0.077%
Allen-Cahn	本文方法 (RFM)	3.149	0.053131	0.052785	0.656%
Allen-Cahn	对比方法 (Deep BSDE)	29.192	0.052846	0.052785	0.116%

在本文实现和参数设置下, RFM 在两个半线性算例中均具有更短的运行时间. 其中 HJB-LQ 算例同时获得更小的初值误差, 而 Allen-Cahn 算例中 Deep BSDE 在更长训练时间下取得更小误差. 因此, 本文实验说明 RFM 在部分 BSDE 离散问题上具有较高计算效率, 但其精度优势依赖具体方程和参数设置. 图 3.6 展示了初值相对误差随时间变化的过程.

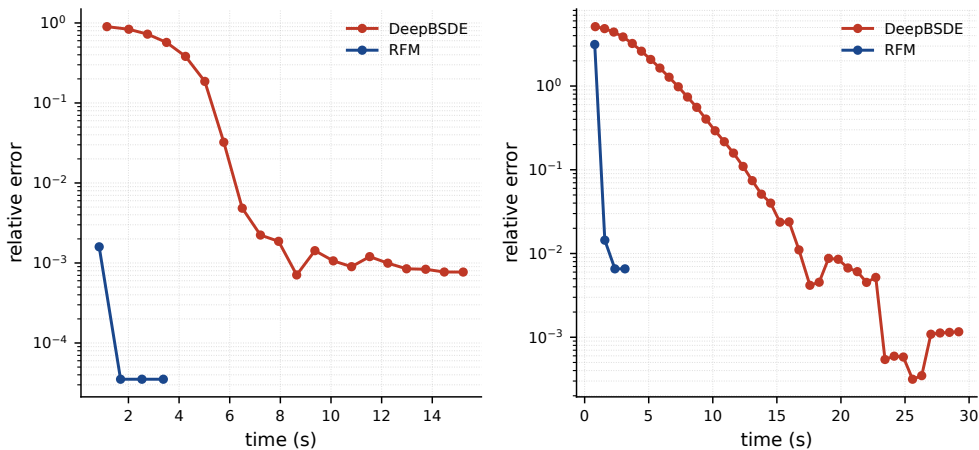


图 3.6 本文方法与 Deep BSDE 对比方法的初值相对误差

第六节 本章小结

本章的四个算例覆盖了线性最小二乘和半线性 LM 两种求解情形. 在线性方程中, Black–Scholes 与 Heat 方程的初值相对误差均低于 0.3%. 在半线性方程中, HJB-LQ 和 Allen–Cahn 也能在标准配置下得到接近参考值的初值. 样本数实验说明, 训练残差过小并不一定代表泛化良好, 因而独立测试残差是必要的观察指标. 维度实验显示, 当前实现在线性方程上的时间增长较慢, 半线性方程的运行时间随维度增长更快; 同时, 高维下仍需要同时观察相对误差和测试残差, 例如 Heat 方程在 $d = 300$ 时出现了较明显的精度退化. 与 Deep BSDE 对比方法的实验表明, 在本文两个半线性算例上, 随机特征方法可以用较少训练时间得到接近参考值的初值.

第四章 总结与展望

本文研究了 BSDE 框架下随机特征方法在高维抛物型 PDE 初值计算中的应用. 这一方法不直接在高维区域上构造网格, 而是从给定初始点出发生成样本路径, 再沿离散 BSDE 推进 Y_k . 其中梯度相关变量 Z_k 由固定内层参数的随机特征函数近似, 需要求解的量只有初值 y_0 和输出层系数 α . 在线性方程中, 终端匹配可以整理成带岭正则化的线性最小二乘问题; 在半线性方程中, 则将终端残差作为非线性最小二乘目标, 并用 LM 方法求解. 这些推导和实现构成了本文算法部分的主要内容.

从数值结果看, 本文方法在热方程, Black-Scholes 方程, HJB-LQ 方程和 Allen-Cahn 方程上都能得到接近参考值的初值. 线性算例说明, 随机特征表示和线性最小二乘求解器可以较好地配合; 半线性算例说明, 对含 z 型或 y 型非线性的方程, LM 迭代也能在本文的参数设置下正常工作. 同时, 参数实验也暴露出一个需要注意的问题: 训练残差小并不一定表示结果可靠. 当样本数偏少时, 模型可能只是较好地拟合了训练路径, 但在独立测试路径上仍有较大的终端残差. 因此, 本文在报告初值误差的同时也给出了测试残差, 以便更全面地观察计算结果. 与 Deep BSDE 方法的对比表明, 随机特征方法在本文测试的半线性算例上运行时间较短, 但不同方程上的精度表现并不完全相同.

当然, 本文的工作还存在一些不足. 随机特征的内层参数在生成后保持固定, 因此近似效果会受到特征宽度, 随机初始化以及样本路径分布的影响. 当特征数不足, 或者最小二乘系统条件较差时, 初值误差和测试残差都可能变大. 对半线性问题而言, LM 方法还需要计算雅可比矩阵并求解阻尼线性系统, 当维度和特征数继续增加时, 显存占用和线性代数开销会成为主要限制. 本文实验也主要集中在几个标准算例上, 对更复杂边界条件, 非光滑终端条件和更强非线性方程的稳定性还没有充分讨论.

后续可以继续从数值稳定性和适用范围两个方面改进. 在随机特征构造上, 可以尝试按时间层调整特征尺度, 或引入结构化随机特征和特征筛选方法, 以改善病态最小二乘系统. 在半线性求解上, 可以考虑无矩阵形式的 LM/Gauss-Newton 近似, 或结合分块求解和混合精度计算来降低显存压力. 此外, 还可以进一步比较 RFM, Deep BSDE 以及其他 BSDE 型算法在不同非线性结构下的误差来源, 分别考察采样误差, 时间离散误差和函数逼近误差的影响. 如果这些问题能够得到更系统的分析, 该方法还有可能推广到更一般的金融定价和随机控制模型中.

参 考 文 献

- [1] Han J, Jentzen A, E W. Solving high-dimensional partial differential equations using deep learning[J]. *Proceedings of the National Academy of Sciences*, 2018, 115(34): 8505-8510.
- [2] E W, Han J, Jentzen A. Algorithms for solving high dimensional PDEs: from nonlinear Monte Carlo to machine learning[J]. *Nonlinearity*, 2022, 35(1): 278-310.
- [3] E W, Yu B. The deep ritz method: a deep learning-based numerical algorithm for solving variational problems[J]. *Communications in Mathematics and Statistics*, 2018, 6(1): 1-12.
- [4] Sirignano J, Spiliopoulos K. DGM: a deep learning algorithm for solving partial differential equations[J]. *Journal of Computational Physics*, 2018, 375: 1339-1364.
- [5] Raissi M, Perdikaris P, Karniadakis G E. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving non-linear partial differential equations[J]. *Journal of Computational Physics*, 2019, 378: 686-707.
- [6] Berg J, Nyström K. A unified deep artificial neural network approach to partial differential equations in complex geometries[J]. *Neurocomputing*, 2018, 317: 28-41.
- [7] Zang Y, Bao G, Ye X, et al. Weak adversarial networks for high-dimensional partial differential equations[J]. *Journal of Computational Physics*, 2020, 411: 109409.
- [8] Kuo F Y, Schwab C, Sloan I H. Quasi-Monte Carlo methods for high-dimensional integration: the standard weighted hilbert space setting and beyond[J]. *The ANZIAM Journal*, 2011, 53(1): 1-37.
- [9] Chen J, Du R, Li P, et al. Quasi-Monte Carlo sampling for solving partial differential equations by deep neural networks[J]. *Numerical Mathematics: Theory, Methods and Applications*, 2021, 14(2): 377-404.
- [10] Hu Z, Shukla K, Karniadakis G E, et al. Tackling the curse of dimensionality with physics-informed neural networks[J]. *Neural Networks*, 2024, 176: 106369.
- [11] Lu L, Meng X, Mao Z, et al. DeepXDE: a deep learning library for solving

- differential equations[J]. SIAM Review, 2021, 63(1): 208-228.
- [12] E W. Machine learning and computational mathematics[J]. Communications in Computational Physics, 2020, 28(5): 1639-1670.
- [13] Beck C, E W, Jentzen A. Machine learning approximation algorithms for high-dimensional fully nonlinear partial differential equations and second-order backward stochastic differential equations[J]. Journal of Nonlinear Science, 2019, 29(4): 1563-1619.
- [14] Black F, Scholes M. The pricing of options and corporate liabilities[J]. Journal of Political Economy, 1973, 81(3): 637-654.
- [15] Hu J, Gan S. High order method for Black–Scholes PDE[J]. Computers & Mathematics with Applications, 2018, 75(7): 2259-2270.
- [16] Zhao T, Sun C, Cohen A, et al. Quantum-inspired variational algorithms for partial differential equations: application to financial derivative pricing[J]. Quantitative Finance, 2024, 24(1): 1-11.
- [17] Chen J, Chi X, E W, et al. Bridging traditional and machine learning-based algorithms for solving PDEs: the random feature method[A/OL]. arXiv (2022). <https://arxiv.org/abs/2207.13380>.
- [18] Wang Y, Dong S. An extreme learning machine-based method for computational PDEs in higher dimensions[A/OL]. arXiv (2023). <https://arxiv.org/abs/2309.07049>.
- [19] Mao T, Xu J, Xu X. Solving high-dimensional PDEs using linearized neural networks[A/OL]. arXiv (2026). <https://arxiv.org/abs/2601.11771>.
- [20] Reid I, Markou S, Choromanski K, et al. Variance-reducing couplings for random features[A/OL]. arXiv (2024). <https://arxiv.org/abs/2405.16541>.
- [21] E W, Han J, Jentzen A. Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations[J]. Communications in Mathematics and Statistics, 2017, 5(4): 349-380.

附录 A 迭代格式推导

本章给出第二章中若干关键公式的详细推导.

第一节 从 PDE 到 BSDE

考虑状态过程 (2.2) 和光滑函数 $u(t, x)$. 由 Itô 公式,

$$\begin{aligned} du(t, X_t) &= \frac{\partial u}{\partial t}(t, X_t) dt + \nabla u(t, X_t)^T dX_t \\ &\quad + \frac{1}{2} \text{Tr}(\sigma \sigma^T(t, X_t) \nabla_x^2 u(t, X_t)) dt. \end{aligned} \quad (\text{A.1})$$

将 (2.2) 代入上式, 得

$$\begin{aligned} du(t, X_t) &= \left(\frac{\partial u}{\partial t}(t, X_t) + \mu(t, X_t) \cdot \nabla u(t, X_t) \right. \\ &\quad \left. + \frac{1}{2} \text{Tr}(\sigma \sigma^T(t, X_t) \nabla_x^2 u(t, X_t)) \right) dt \\ &\quad + \nabla u(t, X_t)^T \sigma(t, X_t) dW_t. \end{aligned} \quad (\text{A.2})$$

若 u 满足方程 (1.2), 则

$$\begin{aligned} \frac{\partial u}{\partial t}(t, x) + \frac{1}{2} \text{Tr}(\sigma \sigma^T(t, x) \nabla_x^2 u(t, x)) \\ + \langle \mu(t, x), \nabla u(t, x) \rangle + f(t, x, u(t, x), \sigma^T(t, x) \nabla u(t, x)) = 0. \end{aligned}$$

代回 (A.2) 得

$$\begin{aligned} du(t, X_t) &= -f(t, X_t, u(t, X_t), \sigma^T(t, X_t) \nabla u(t, X_t)) dt \\ &\quad + \nabla u(t, X_t)^T \sigma(t, X_t) dW_t. \end{aligned} \quad (\text{A.3})$$

积分形式为

$$\begin{aligned} u(t, X_t) - u(0, X_0) &= - \int_0^t f(s, X_s, u(s, X_s), \sigma^T(s, X_s) \nabla u(s, X_s)) ds \\ &\quad + \int_0^t \nabla u(s, X_s)^T \sigma(s, X_s) dW_s. \end{aligned} \quad (\text{A.4})$$

再利用 (2.3) 的定义, 就得到正文中的 BSDE (2.4).

第二节 线性方程终端格式的展开

对线性生成元, 离散迭代式 (2.8) 可写为

$$y_{k+1}^{(i)} = a_k^{(i)} y_k^{(i)} + (\xi_k^{(i)})^T \alpha H_k^{(i)} + c_k^{(i)}, \quad (\text{A.5})$$

其中

$$a_k^{(i)} = 1 - \Delta t \mathcal{L}(t_k, x_k^{(i)}), \quad (\text{A.6})$$

$$\xi_k^{(i)} = \Delta W_k^{(i)}, \quad (\text{A.7})$$

$$c_k^{(i)} = -\Delta t \mathcal{N}(t_k, x_k^{(i)}). \quad (\text{A.8})$$

从 $y_0^{(i)} = y_0$ 出发, 前几步展开为

$$y_1^{(i)} = a_0^{(i)} y_0 + (\xi_0^{(i)})^T \alpha H_0^{(i)} + c_0^{(i)}, \quad (\text{A.9})$$

$$\begin{aligned} y_2^{(i)} &= a_1^{(i)} y_1^{(i)} + (\xi_1^{(i)})^T \alpha H_1^{(i)} + c_1^{(i)} \\ &= a_1^{(i)} a_0^{(i)} y_0 + a_1^{(i)} (\xi_0^{(i)})^T \alpha H_0^{(i)} + (\xi_1^{(i)})^T \alpha H_1^{(i)} \\ &\quad + a_1^{(i)} c_0^{(i)} + c_1^{(i)}. \end{aligned} \quad (\text{A.10})$$

由此可见, 每一步都会把已有项乘上新的系数 $a_k^{(i)}$, 再加上当前时刻的随机特征项和常数项. 归纳可得

$$\begin{aligned} y_N^{(i)} &= \left(\prod_{j=0}^{N-1} a_j^{(i)} \right) y_0 \\ &\quad + \left\langle \sum_{k=0}^{N-1} \left(\prod_{j=k+1}^{N-1} a_j^{(i)} \right) \xi_k^{(i)} (H_k^{(i)})^T, \alpha \right\rangle \\ &\quad + \sum_{k=0}^{N-1} \left(\prod_{j=k+1}^{N-1} a_j^{(i)} \right) c_k^{(i)}. \end{aligned} \quad (\text{A.11})$$

这里约定当 $k = N - 1$ 时, 空乘积取为 1.

将 (A.11) 与正文中的 (2.9) 对照, 可直接识别

$$\beta_i = \prod_{j=0}^{N-1} a_j^{(i)}, \quad (\text{A.12})$$

$$\Gamma_i = \sum_{k=0}^{N-1} \left(\prod_{j=k+1}^{N-1} a_j^{(i)} \right) \xi_k^{(i)} (H_k^{(i)})^T, \quad (\text{A.13})$$

$$\eta_i = \sum_{k=0}^{N-1} \left(\prod_{j=k+1}^{N-1} a_j^{(i)} \right) c_k^{(i)}. \quad (\text{A.14})$$

因而线性情形确实归结为关于 y_0 和 α 的线性最小二乘问题.

第三节 半线性方程的灵敏度递推

对半线性情形, 令

$$z_k^{(i)} = \alpha H_k^{(i)}.$$

由于参数向量取为 (2.17), 故

$$\frac{\partial z_k^{(i)}}{\partial \theta} = \begin{bmatrix} 0_{d \times 1} & I_d \otimes (H_k^{(i)})^T \end{bmatrix}. \quad (\text{A.15})$$

记

$$S_k^{(i)} = \frac{\partial y_k^{(i)}}{\partial \theta}.$$

因为 y_0 是参数向量的第一项, 而其余参数在初始时刻不直接进入 y_0 , 所以有初值 (2.26) .

对离散迭代式 (2.16) 关于 θ 求导:

$$\begin{aligned} S_{k+1}^{(i)} &= S_k^{(i)} - \Delta t \frac{\partial}{\partial \theta} f(t_k, x_k^{(i)}, y_k^{(i)}, z_k^{(i)}) \\ &\quad + (\Delta W_k^{(i)})^T \frac{\partial z_k^{(i)}}{\partial \theta}. \end{aligned}$$

对生成元使用链式法则,

$$\frac{\partial}{\partial \theta} f(t_k, x_k^{(i)}, y_k^{(i)}, z_k^{(i)}) = f_{y,k}^{(i)} S_k^{(i)} + f_{z,k}^{(i)} \frac{\partial z_k^{(i)}}{\partial \theta},$$

其中 $f_{y,k}^{(i)}$ 和 $f_{z,k}^{(i)}$ 的定义见 (2.28) . 代回上式得

$$\begin{aligned} S_{k+1}^{(i)} &= \left(1 - \Delta t f_{y,k}^{(i)}\right) S_k^{(i)} \\ &\quad + \left((\Delta W_k^{(i)})^T - \Delta t f_{z,k}^{(i)}\right) \frac{\partial z_k^{(i)}}{\partial \theta}. \end{aligned} \quad (\text{A.16})$$

再代入 (A.15) , 即得到正文中的递推式 (2.27) . 记

$$A_k^{(i)} = 1 - \Delta t f_{y,k}^{(i)}, \quad (\text{A.17})$$

$$B_k^{(i)} = ((\Delta W_k^{(i)})^T - \Delta t f_{z,k}^{(i)}) \begin{bmatrix} 0_{d \times 1} & I_d \otimes (H_k^{(i)})^T \end{bmatrix}. \quad (\text{A.18})$$

则

$$S_{k+1}^{(i)} = A_k^{(i)} S_k^{(i)} + B_k^{(i)}. \quad (\text{A.19})$$

展开可得

$$S_N^{(i)} = \left(\prod_{j=0}^{N-1} A_j^{(i)}\right) S_0^{(i)} + \sum_{k=0}^{N-1} \left(\prod_{j=k+1}^{N-1} A_j^{(i)}\right) B_k^{(i)}. \quad (\text{A.20})$$

由于

$$r_i(\theta) = y_N^{(i)}(\theta) - g(x_N^{(i)}),$$

而终端状态 $x_N^{(i)}$ 与参数 θ 无关, 故

$$\frac{\partial r_i}{\partial \theta} = \frac{\partial y_N^{(i)}}{\partial \theta} = S_N^{(i)}.$$

这就说明了雅可比矩阵可以由所有路径的终端灵敏度直接拼接得到.

附录 B 参考值的计算与可靠性说明

本附录说明第三章中各算例参考值的计算方法. 对具有显式解的方程, 直接使用解析公式. 对没有简单闭式解的方程, 使用与正文 RFM 求解器无关的数值方法计算参考值, 并给出相应的置信区间或误差估计.

第一节 Heat 方程的显式参考值

Heat 方程的终端条件取为

$$g(x) = \exp\left(-\frac{\|x\|^2}{d}\right), \quad x_0 = 0, \quad T = 1. \quad (\text{B.1})$$

更一般地, 若终端条件写为

$$g(x) = \exp\left(-\frac{\|x\|^2}{a}\right),$$

则由 Feynman-Kac 公式可得

$$u(t, x) = \left(\frac{a}{a + 2(T - t)}\right)^{d/2} \exp\left(-\frac{\|x\|^2}{a + 2(T - t)}\right). \quad (\text{B.2})$$

因而在 $x_0 = 0, a = d$ 时,

$$Y_0 = \left(\frac{d}{d + 2}\right)^{d/2}. \quad (\text{B.3})$$

代入不同维度得到表 B.1. 这些值由显式公式直接计算, 不包含额外的数值离散误差.

表 B.1 Heat 方程参考值

d	Y_0
20	0.385543289429532
30	0.379812405815246
50	0.375116802253964
100	0.371527882126961
200	0.369711212329119
300	0.369102310996184

第二节 HJB-LQ 方程的积分参考值

HJB-LQ 方程的终端条件取为

$$g(x) = \log\left(\frac{1 + \|x\|^2}{2}\right). \quad (\text{B.4})$$

HJB-LQ 方程对应的 BSDE 为

$$dY_t = \frac{1}{2} \|Z_t\|^2 dt + Z_t^T dW_t. \quad (\text{B.5})$$

令 $U_t = \exp(-Y_t)$, 则 U_t 为鞅. 于是

$$Y_0 = -\log \mathbb{E} \left[\frac{2}{1 + 2T\chi_d^2} \right]. \quad (\text{B.6})$$

因此参考值可以通过下面的一维积分计算:

$$\mathbb{E} \left[\frac{2}{1 + 2T\chi_d^2} \right] = \int_0^\infty \frac{2}{1 + 2Ty} f_{\chi_d^2}(y) dy. \quad (\text{B.7})$$

本文采用自适应 Simpson 积分在有限区间上计算该积分. 当积分上限从 300 增加到 500 时, 结果变化不超过 5×10^{-12} , 因而在本文所列精度下, 截断误差可以忽略. 最终结果见表 B.2.

表 B.2 HJB-LQ 方程参考值

d	Y_0
20	2.921014735738689
30	3.351223237182463
50	3.882006682195226
100	4.590161724604434
200	5.290814736008237
300	5.698781231260821

第三节 Black-Scholes 方程的 Monte Carlo 参考值

对 Black-Scholes 方程, 终端收益函数为高维算术平均型欧式看涨期权

$$g(x) = \max \left(\frac{1}{d} \sum_{m=1}^d x_m - K, 0 \right). \quad (\text{B.8})$$

经典一维欧式期权定价已经有成熟的 Black-Scholes 基本方程和闭式公式^[14]. 但这里的算术平均收益依赖多个标的资产, 因而该问题没有类似一维 Black-Scholes 公式的简单闭式解. 因此本文采用独立的大样本 Monte Carlo 方法估计其初值, 并使用几何平均期权的闭式定价公式作为控制变量.

记算术平均收益对应的贴现随机变量为 A , 几何平均收益对应的贴现随机变量为 G . 由于几何平均

$$\bar{G}_T = \exp \left(\frac{1}{d} \sum_{m=1}^d \log X_T^m \right), \quad (\text{B.9})$$

仍服从对数正态分布, 因而 $\mathbb{E}[G]$ 可以显式计算. 于是采用控制变量估计量

$$A^* = A - \beta(G - \mathbb{E}[G]), \quad (\text{B.10})$$

同时结合对偶路径进一步降低方差.

本文使用 10^6 条路径计算参考值, 结果和 95% 置信区间见表 B.3.

表 B.3 Black–Scholes 方程 Monte Carlo 参考值

d	Y_0	95% 置信区间
20	0.05173661	[0.05172011, 0.05175310]
30	0.05021835	[0.05020434, 0.05023235]
50	0.04921751	[0.04920643, 0.04922860]
100	0.04881270	[0.04880488, 0.04882053]
200	0.04877301	[0.04876747, 0.04877854]
300	0.04876993	[0.04876541, 0.04877445]

第四节 Allen–Cahn 方程的有限差分参考值

对 Allen–Cahn 方程, 由于其终端条件和扩散过程都具有径向对称性, 该问题可以转化为径向变量上的一维半线性抛物方程. 记 $r = \|x\|$, 则解可写为 $u(t, x) = v(t, r)$. 在反向时间变量 $\tau = T - t$ 下, 参考解满足

$$\frac{\partial v}{\partial \tau} = \frac{\partial^2 v}{\partial r^2} + \frac{d-1}{r} \frac{\partial v}{\partial r} + v - v^3, \quad v(0, r) = \frac{1}{2 + 0.4r^2}. \quad (\text{B.11})$$

Allen–Cahn 方程是高维半线性 PDE 算法中常用的反应–扩散型测试问题^[1]. 本文在区间 $r \in [0, 20]$ 上采用径向有限差分离散, 对扩散项作隐式处理, 对非线性反应项作显式处理. 在 $r = 0$ 处施加径向对称边界条件, 在远端边界取零值近似.

为评估离散误差, 分别使用两组网格

$$(\Delta r, \Delta \tau) = (0.025, 5 \times 10^{-5}),$$

$$(\Delta r, \Delta \tau) = (0.0125, 2.5 \times 10^{-5})$$

进行计算, 并将两组结果的差作为误差估计. 最终采用细网格结果作为参考值, 结果见表 B.4.

此外, 当边界截断半径从 15 增加到 30 时结果几乎不变, 因而边界截断误差小于当前网格误差.

表 B.4 Allen–Cahn 方程有限差分参考值

d	Y_0	网格差估计
20	0.20635880	2.08×10^{-6}
30	0.15199385	1.59×10^{-6}
50	0.09908593	3.19×10^{-6}
100	0.05278464	2.81×10^{-6}
200	0.02724542	1.79×10^{-6}
300	0.01835709	1.29×10^{-6}

致 谢

在撰写毕业论文期间,我有幸得到中国科大苏州高研院 SCAI 实验室陈景润教授和孙羿斐博士的教导.两位老师不仅在研究思路和具体问题上给予我指导,也在科研态度和人生规划上对我产生了深远影响.在此谨向他们的悉心教导和帮助致以诚挚的感谢.

我对数学的热爱始于高中时期.在 17 岁以前,我一直秉持实用主义的想法,认为物理比数学更具有现实价值.那时我立志成为邓稼先那样推动我国科技进步的人.然而少年时期的心境常常会因一些偶然的契机而改变.17 岁生日时,一位同学送给我一本数学科普读物万物皆数,由此引发了我对数学的兴趣.这本书聚焦于数学的实用价值和美学价值,不仅打破了我对数学只是抽象工具且缺少实际应用的刻板印象,也让我第一次真切地感受到数学的优雅.尽管它并没有直接促使我选择数学专业,却在我心中埋下了接纳数学的种子.因此,我也感谢这位同学和这本书对我的影响.

我还要感谢父母和高中时期的老师们.由于学校课业繁重,又有未来高考的压力,我常常睡眠不足,上课时有时会不由自主地打盹.我的班主任兼数学老师甄健华老师发现后,并没有责备我,而是与我和家长沟通,一同寻找解决办法.最终,父母在学校附近租下一套房子,尽力保证我中午和晚上的休息.除班主任外,其他科目的老师也都认真负责.语文老师奉行“学科素养教育”,培养了我对文学和美学的热爱.英语老师授课严谨而认真,在我考上中国科大的少创班后,仍提醒我不能放下英语学习.作为物理课代表,我与物理老师的互动较多,她是一位认真负责而善良的老师.有时考虑到我们的作业较多,她会减少作业量或延缓作业提交时间.在我离开高中时,物理老师也因病需要治疗而暂别讲台,所幸后来身体渐渐恢复,又回到了教书育人的岗位.化学老师幽默风趣,课堂上也常向我们分享成长经验,在一定程度上塑造了我的人格.生物老师平易近人,与学生关系融洽,专业知识也十分丰富.

中国科大的少年创新班给了我新的机遇.我的语文和英语成绩一直不算理想,也曾常常担心自己会因偏科而无法进入喜欢的学校,学习感兴趣的专业.中国科大的少创班给了我展示自己的机会.我十分感谢科大提供的平台,使我能够心无旁骛地学习和科研.

最后,我要感谢我身边的朋友们.他们与我一起学习,一起娱乐,也在我遇到困难时伸出援手.本科生活中的许多乐趣,都来自他们与我共同分享的快乐.

2026 年 5 月

实验算法伪代码

本附录给出本文实验中实际采用的求解伪代码. 为了便于与正文中的方法分析对应, 这里不再强调具体实现细节, 而是统一使用“路径采样, 特征构造, 参数求解, 测试评估”的算法表述.

第五节 线性方程求解流程

算法 B.1 线性方程的求解流程**Data:** 训练路径, 布朗增量, 随机特征, 线性方程离散系数, 岭正则化参数 λ_r **Result:** y_0, α

- 1 对每条路径计算终端表达式中的系数 $\beta_i, \Gamma_i, \eta_i$;
- 2 将所有路径拼接为设计矩阵 A 与观测向量 B ;
- 3 令参数向量

$$\vartheta = \begin{bmatrix} y_0 \\ \text{vec}(\alpha) \end{bmatrix},$$

并记 $p = 1 + dH$;

- 4 构造

$$R = \begin{bmatrix} 0 & 0 \\ 0 & I_{dH} \end{bmatrix}$$

用于仅对 α 正则化;

- 5 **if** $S \geq p$ **then**

- 6 求解原始岭正则化系统

$$(A^\top A + \lambda_r R^\top R) \vartheta = A^\top B$$

得到参数 θ ;

- 7 **else**

- 8 求解对偶岭正则化系统

$$\vartheta = A^\top (AA^\top + \lambda_r I_S)^{-1} B$$

得到参数 θ ;

- 9 **end**

- 10 解包 θ 得到参数 y_0 与 α ;

- 11 返回 y_0 与 α ;

第六节 半线性方程求解流程

算法 B.2 半线性方程的 LM 求解流程**Data:** 训练路径, 布朗增量, 随机特征, 初始参数 $\theta^{(0)}$, 初始阻尼参数 $\lambda^{(0)}$ **Result:** y_0, α

```

1  令  $m \leftarrow 0$ ;
2  while 未满足停止准则 do
3      根据当前参数  $\theta^{(m)}$  构造所有  $Z_k^{(i)}$ ;
4      前向推进离散 BSDE, 计算终端残差向量  $r(\theta^{(m)})$ ;
5      计算雅可比矩阵  $J(\theta^{(m)})$ ;
6      求解阻尼子问题
          
$$(J^\top J + \lambda^{(m)} I) \delta^{(m)} = -J^\top r;$$

          令试探参数  $\theta_{\text{trial}} = \theta^{(m)} + \delta^{(m)}$ ;
7      计算试探参数对应的目标函数值;
8      if 目标函数下降 then
9          接受该步, 令  $\theta^{(m+1)} = \theta_{\text{trial}}$ ;
10         适当减小阻尼参数  $\lambda^{(m)}$ ;
11          $m \leftarrow m + 1$ ;
12     else
13         拒绝该步并增大阻尼参数, 重新计算当前迭代的试探步;
14     end
15 end
16 从最终参数向量中拆分出  $y_0$  与  $\alpha$ ;
17 返回  $y_0$  与  $\alpha$ ;

```

第七节 测试误差评估流程

算法 B.3 测试误差评估流程

Data: 训练得到的 y_0, α

Result: 独立测试样本上的终端残差 RMSE

- 1 重新生成独立测试路径 $\{X_k^{(i)}, \Delta W_k^{(i)}\}$;
- 2 在测试路径上构造随机特征 $\{H_k^{(i)}\}$;
- 3 用固定参数 α 构造 $Z_k^{(i)} = \alpha H_k^{(i)}$;
- 4 沿测试路径前向推进离散 BSDE, 得到所有终端值 $Y_N^{(i)}$;
- 5 计算

$$\text{RMSE}_{\text{test}} = \sqrt{\frac{1}{S} \sum_{i=1}^S (Y_N^{(i)} - g(X_N^{(i)}))^2};$$

返回测试 RMSE;
